

Concurrencia (1er. parcial, recuperación) (Solución)

Curso 2025–2026 - 2º semestre (junio 2026)

Grado en Ingeniería Informática / Grado en Matemáticas e Informática / 2ble. grado en Ing. Informática y ADE

UNIVERSIDAD POLITÉCNICA DE MADRID

NORMAS: Este es un cuestionario que consta de 9 preguntas. Todas las preguntas son de respuesta simple excepto la pregunta 9 que es una pregunta de desarrollo. La puntuación total del examen es de 10 puntos. La duración total es de **una hora. Sólo hay una respuesta válida a cada pregunta de respuesta simple.** Toda pregunta en que se marque más de una respuesta se considerará incorrectamente contestada. Las respuestas deben consignarse en el casillero de la derecha. Toda **pregunta incorrectamente contestada restará** del examen una cantidad de puntos igual a la puntuación de la pregunta dividida por el número de alternativas ofrecidas en la misma. No olvidéis **rellenar vuestros datos.**

RESPUESTAS

1	2	3	4
5	6	7	8

Cuestionario

- (1 punto) 1. A continuación se muestra un algoritmo de exclusión mutua para dos procesos usando espera activa.

volatile int vez1 = 0, vez2 = 0;	
<pre> class T1 extends Thread { public void run() { vez1 = vez2+1; while (vez2 != 0 && vez1 >= vez2) { } seccion_critical(); vez1 = 0; seccion_no_critical(); } } </pre>	<pre> class T2 extends Thread { public void run() { vez2 = vez1 + 1; while (vez1 != 0 && vez2 > vez1) { } seccion_critica2(); vez2 = 0; seccion_no_critica2(); } } </pre>

Se pide marcar la afirmación correcta:

- (a) ☐ El algoritmo no garantiza la exclusión mutua en el acceso a las secciones críticas.
 (b) ☒ El algoritmo garantiza la exclusión mutua en el acceso a las secciones críticas.

- (1 punto) 2. Dado el siguiente programa concurrente (ignorar la falta de captura de excepciones):

<pre> class A extends Thread { public void run() { Thread b1 = new B(), b2 = new B(); b1.start(); b2.start(); b1.join(); b2.join(); System.out.print("A"); } } class B extends Thread { public void run() { System.out.print("B"); } } </pre>	<pre> class Threads { public static void main(String[] as) { Thread a1 = new A(), a2 = new A(); a1.start(); a2.start(); } } </pre>
---	--

Se quiere estudiar el comportamiento del programa tal cual está y compararlo con una versión donde se han eliminado los join de la clase A que están enmarcados. Se pide marcar la afirmación correcta:

- (a) ☒ Con los join, cada A aparece en la salida después de las dos B creadas por ese mismo hilo.
 (b) ☐ Sin los join, cada A aparece en la salida después de las dos B creadas por ese mismo hilo.
 (c) ☐ El comportamiento es igual con y sin los join

(1 punto) 3. Dado el programa de la pregunta 2 (manteniendo los join).

Se pide marcar cuál es el número máximo de hilos que podrían estar en ejecución simultáneamente.

- (a) ☐ 3
 (b) ☒ 7

(1 punto) 4. **Se pide** marcar la afirmación correcta respecto a sentencias atómicas en Java:

- (a) ☐ El incremento `x++;` de una variable entera es una acción atómica.
 (b) ☐ La negación `b=!b;` de una variable booleana es una acción atómica.
 (c) ☒ Ninguna de las anteriores.

(1 punto) 5. Dado un programa concurrente con tres procesos instancias de P1, P2 y P3:

<pre>volatile int x = 1; Semaphore s1 = new Semaphore (2); Semaphore s2 = new Semaphore (1); Semaphore s3 = new Semaphore (0);</pre>		
<pre>class P1 { void run() { s2.await(); s1.await(); x *= 3; s3.signal (); } }</pre>	<pre>class P2 { void run() { s3.await(); x += 2; s1.signal (); } }</pre>	<pre>class P3 { void run() { s1.await(); x += 3; s1.signal(); } }</pre>

Se pide **marcar** cuál será el valor de la variable x al acabar el programa.

- (a) ☐ 8.
 (b) ☐ Puede ser 11 o bien 8.
 (c) ☒ No se puede saber porque el programa tiene una condición de carrera.

(1 punto) 6. Dado el siguiente programa concurrente:

<pre>class Filo extends Thread { private Semaphore l,r; public Filo(Semaphore l, Semaphore r) { this.l = l; this.r = r; } public void run() { while (true) { System.out.println("Pensando"); l.await(); r.await(); System.out.println("Comiendo"); l.signal(); r.signal(); } } }</pre>	<pre>class Filosofos { public static void main(String[] as) { Semaphore[] s = new Semaphore[5]; for (int i = 0; i < 5; i++) { s[i] = new Semaphore(1); } for (int i = 0; i < 5; i++) { new Filo(s[i], s[(i+1)%5]).start(); } } }</pre>
--	--

Se pide marcar la afirmación correcta:

- (a) ☒ El programa no cumple la propiedad de ausencia de interbloqueo (puede sufrir un interbloqueo).
 (b) ☐ El programa cumple la propiedad de ausencia de interbloqueo (no puede sufrir un interbloqueo).

- (1 punto) 7. Se tiene un CTAD que almacena las existencias (e y el año de caducidad c) de una serie de productos identificados con un número entre 1 y 100. A continuación mostramos parte de la especificación, que incluye la POST de una acción llamada o . (Hemos usado la notación s_i para referirnos al i -ésimo elemento de la secuencia s y $x \in s$ para indicar que x está dentro de la secuencia s).

C-TAD Almacen

....

ACCIÓN o : **Secuencia**($\mathbb{N}$)[s]

....

TIPO Almacen = Clave $\rightarrow (e: \mathbb{N} \times c: \mathbb{N})$

DONDE Clave = $\{1 \dots 100\}$

....

$o(s)$

POST: $\forall i \in \{1 \dots \text{Longitud}(s)\} . \text{self}(s(i)).c < 2026 \wedge \forall j \in \text{Clave} . \text{self}(j).c < 2026 \rightarrow j \in s$

¿Qué se ha indicado que calcula la acción o ?

- (a) ☐ Devuelve las claves de los productos de los que hay más de 2026 existencias.
 (b) ☐ Devuelve las claves de los productos de los que hay menos de 2026 existencias.
 (c) ☒ Devuelve las claves de los productos que SÍ han caducado.
 (d) ☐ Devuelve las claves de los productos que NO han caducado.

- (1 punto) 8. Supongamos un programa concurrente con dos procesos y ninguno más usando el recurso Auth: un proceso de interfaz de usuario que en bucle se dedica a actualizar el nombre de usuario y la clave (mediante la operación `formulario`) y otro proceso que en bucle se encarga de leer el nombre de usuario y la clave y luego autorizarlo si la clave es correcta (mediante las operaciones `leer` y `autorizar`).

C-TAD Auth

OPERACIONES

ACCIÓN `formulario`: **String**[e] $\times$ **String**[e]

ACCIÓN `leer`: **String**[s] $\times$ **String**[s]

ACCIÓN `autorizar`: **String**[e] $\times$ **String**[e]

SEMÁNTICA

DOMINIO

TIPO Auth = (u : **String** $\times$
 c : **String** $\times$
 $leido$: $\mathbb{B}$ $\times$
 $autorizado$: $\mathbb{B}$)

INICIAL $\neg \text{self}.leido \wedge \neg \text{self}.autorizado$

CPRE: $\neg \text{self}.autorizado$

`formulario`(u, c)

POST: $\text{self} = (u, c, \text{falso}, \text{falso})$

CPRE: $\neg \text{self}.leido$

`leer`(u, c)

POST: $\text{self} = (\text{self}^{\text{pre}}.u, \text{self}^{\text{pre}}.c, \text{cierto}, \text{falso})$
 $\wedge u = \text{self}.u \wedge c = \text{self}.c$

CPRE: $\neg \text{self}.autorizado$

`autorizar`(u, c)

POST: $\text{self} = (\text{self}^{\text{pre}}.u,$
 $\text{self}^{\text{pre}}.c,$
 $\text{self}^{\text{pre}}.leido,$
 $\text{self}^{\text{pre}}.u = u \wedge \text{self}^{\text{pre}}.c = c)$

Se pide señalar la respuesta correcta.

- (a) ☐ El estado nunca puede cumplir ($\neg \text{self}.leido \wedge \text{self}.autorizado$).
 (b) ☒ El estado puede llegar a cumplir ($\neg \text{self}.leido \wedge \text{self}.autorizado$).

- (2 puntos) 9. Se está desarrollando una IA para competir en el campeonato del mundo del popular juego de mesa *Los nómadas que cantan*. Los desarrolladores han ideado un recurso compartido para representar las materias primas que tiene el jugador, siendo estas materias primas cereal, agua y madera.

Hay una operación no bloqueante para cargar materias primas: **cargar** con la materia prima como parámetro. El jugador puede llevar a lo sumo una unidad de cada materia prima pero no puede llevar cereal y madera a la vez, si carga una pierde la otra.

Por otro lado, hay dos operaciones que consumen materias primas: **avanzar** (consume cereal y agua) y **reparar** (consume agua y madera). Estas operaciones son bloqueantes hasta que se disponga de las materias primas necesarias.

Se pide: completar la especificación del recurso compartido. Como se puede observar el dominio es un conjunto de materias primas, por lo tanto las operaciones básicas que puedes usar sobre el estado del recurso **self** son la unión ($\cup$), la intersección ($\cap$) y la diferencia de conjuntos ($\setminus$) así como la pertenencia ($\in$) y la inclusión ($\subseteq$).

C-TAD Nomada

OPERACIONES

ACCIÓN cargar: $\text{Materia}[e]$

ACCIÓN avanzar

ACCIÓN reparar

SEMÁNTICA

DOMINIO

TIPO Materia = $\{C, A, M\}$

TIPO Nomada = **Conjunto**(Materia)

INICIAL self = $\{\}$

[El enunciado exige tratar **self** como un conjunto de materias, cualquier otro dominio (productos cartesianos o registros, funciones parciales, etc.) se han considerado incorrectos.]

CPRE: cierto

cargar(m)

POST: $(m = C \Rightarrow \text{self} = \text{self}^{\text{pre}} \cup \{C\} \setminus \{M\})$
 $\wedge (m = M \Rightarrow \text{self} = \text{self}^{\text{pre}} \cup \{M\} \setminus \{C\})$
 $\wedge (m = A \Rightarrow \text{self} = \text{self}^{\text{pre}} \cup \{A\})$

CPRE: $\{C, A\} \subseteq \text{self}$

avanzar

POST: $\text{self} = \text{self}^{\text{pre}} \setminus \{C, A\}$

CPRE: $\{M, A\} \subseteq \text{self}$

reparar

POST: $\text{self} = \text{self}^{\text{pre}} \setminus \{M, A\}$